ALExRS Robotic Rehabilitation: Interim Report

Jia Bhargava
Semester Internship (Spring 2026)

CREO Lab: Advanced Robotics and Human-Centred Technologies
Università Campus Bio-Medico di Roma (UCBM)
Mentors: Prof. Francesco Scotto di Luzio & Prof. Francesca Cordella

I. Problem Statement

The objective of this project is to develop analytical methods for rehabilitation movement monitoring using ALExRS (Arm Light Exoskeleton Rehabilitation Station) designed by Wearable Robotics for upper-limb rehabilitation assistance.

The study focuses on decomposing complex rehabilitation exercises into a sequence of meaningful functional states describing different phases of activity. The identified states are designed to be time-agnostic and trajectory-independent. This means that states are not characterised by the duration of movement or the path followed by the end-effector.

To identify transitions between movement states, both supervised and unsupervised learning techniques are explored. The analysis is performed using kinematic information obtained from the exoskeleton system, including joint angular positions and velocities as well as coordinates of the end-effector.

Detecting state transitions is particularly useful for implementing custom control algorithms for each movement state, enabling more precise assistance and improved safety.

II. Background Research

This project is conducted under the supervision of Francesco Scotto di Luzio and Francesca Cordella, while assisting a masters student, Marco Russo, involving rehabilitation control strategies using the ALExRS.

Background research focuses on understanding the motion representation and control characteristics of the ALExRS. The exoskeleton consists of 8 joint rotations and end effector positons and velocities. The system represents joint angle and positional data with a Cartesian coordinate reference frame. The X-axis is oriented toward the right, the Y-axis is oriented upward, and the Z-axis is oriented backward. For each robotic arm, the origin of the coordinate system is located at the first shoulder joint. The ALExRS exoskeleton along with its coordinate directions is depicted in Fig. 1.

The ALExRS system provides six degrees of freedom (DoF), of which four joints are actively actuated. The actuated joints include shoulder abduction/adduction, shoulder internal/external rotation, shoulder flexion/extension, and elbow flexion/extension. Wrist flexion/extension and forearm pronation/supination are passive. The anatomical planes associated with each movement are illustrated in Fig. 2.

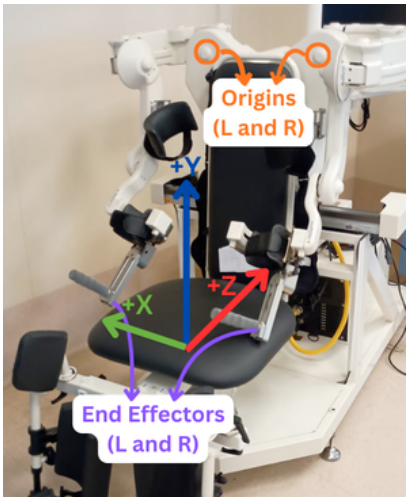

Fig. 1: ALExRS coordinate axes, origins and end-effectors

In the work associated with Marco Russo, multi-state rehabilitation tasks were analysed. Complex rehabilitation exercises were decomposed into discrete functional movement phases to enable adaptive control. The bilateral control mode was used, where one arm (master) moves freely while the other arm (slave) mirrors the movement through robotic actuation. This configuration supports rehabilitation training for patients with unilateral motor impairment by allowing movement assistance through the unaffected limb.

The rehabilitation tasks considered in this study typically involve 5 functional movement phases:

1) REACHING: Moving towards the target object
2) GRASP: Grasping the object through flexion
3) HOLD LIFT: Holding and moving the object upward
4) RELEASE: Releasing the object through extension
5) TURN BACK: Returning to the starting position

The grasping and release actions are supported using functional electrical stimulation (FES) [1] as wrist and finger motions are not actuated by the mechanical system of the exoskeleton. Electromyography (EMG) signals are collected from the master arm to estimate voluntary muscle activation.

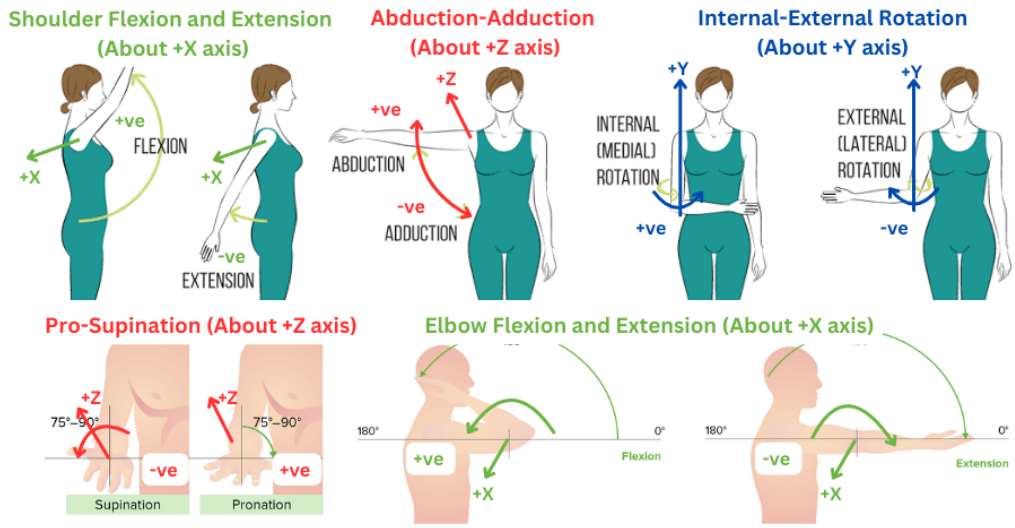

Fig. 2: Arm movements supported by ALExRS

These signals are then used to drive stimulation electrodes that activate corresponding muscles in the slave arm, enabling mirrored movement reproduction. It is evident that the detection of state changes, especially for grasp and release states, is essential to determine the point in time, duration, and muscle channel for applying FES.

Table I details the six muscle channels that are considered for grasping and release assistance.

Name	**Movement**	**Grasp / Release**	**Location**
Flexor Digitorum Superficialis (FDS)	Finger flexion	Grasp	Anterior and ulnar side; Between elbow and wrist
Flexor Carpi Radialis (FCR)	Wrist flexion	Grasp	Anterior and radial side; Closer to elbow
Abductor Pollicis Brevis (APB)	Thumb opposition and abduction	Grasp	Anterior and radial side; Base of thumb near wrist
Extensor Pollicis Longus (EPL)	Thumb extension	Release	Posterior and ulnar side; Closer to wrist
Extensor Digitorum Communis (EDC)	Finger extension	Release	Posterior and central; Between elbow and wrist
Extensor Carpi Ulnaris (ECU)	Wrist extension	Release	Posterior and central; Closer to elbow

TABLE I: Muscle channels used for grasp and release with EMG sensing and FES stimulation.

Hybrid robotic rehabilitation integrates mechanical assistance from the exoskeleton with biological signal-driven control mechanisms [2]. This research investigates such hybrid rehabilitation approaches by combining FES actuation with mechanical exoskeleton movement. This allows the system to replicate a functional task involving reaching, grasping and releasing motions.

III. Methodology and Preliminary Findings

A. Graphical User Interface

The first part of this project involved developing a Windows Forms Graphical User Interface (GUI) that provides real-time monitoring and control of the ALExRS system. The interface is designed to support data acquisition, robot state visualization, and rehabilitation phase management.

The GUI logs and updates data at frequency of approximately 50 Hz, enabling smooth visualization of biomechanical parameters. Joint position, velocity, and phase state information are stored in CSV log files along with their timestamp for subsequent analysis.

The interface supports a preview mode when the robot is not connected and automatically transitions to connected mode when shared memory communication is established. When connected, it displays live kinematic data for each joint and arm. Control panels allow switching between wearing mode, rehabilitation mode and emergency device shutdown. The GUI also implements colour-coded status indicators to distinguish operational modes of the left and right robotic arms.

Figure 3 shows a preview of the GUI interface.

B. Data Acquisition

The preliminary objective of this project was to design and record a simple mono-lateral, point-to-point task that is independent of trajectory and timing variations. The task was structured into two distinct states: moving toward the target (GOING state) and returning to the starting position (RETURNING state). The dataset was collected such that a single transition point between these two states could be identified.

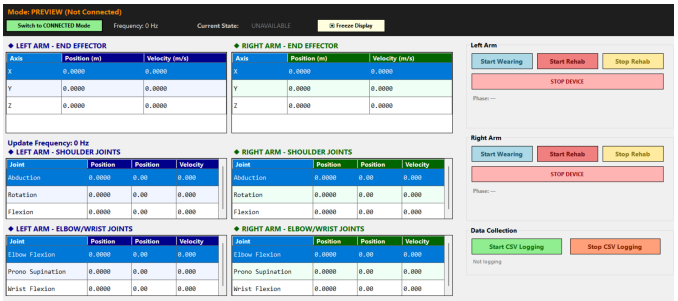

Fig. 3: GUI interface in preview mode

Since training machine learning models requires a sufficiently large dataset, 87 labelled samples were collected. Each sample consisted of approximately 500 time steps. To improve model generalisation, the recorded samples incorporated multiple combinations of movement speeds and trajectories during both the GOING and RETURNING phases.

During data collection, each time step was assigned a state label corresponding to the participant's movement phase. At the beginning of each recording session, data samples were labelled as GOING. When the participant transitioned to the next movement phase by pressing the NEXT STATE button in the GUI, subsequent data points were labelled as RETURNING.

The collected dataset is used for change point detection analysis using both supervised and unsupervised learning approaches.

C. Implementing Change Point Detection Models

The next stage of this project focuses on applying Change Point Detection (CPD) methods to identify a single state transition between the GOING and RETURNING states in the previously acquired dataset. Supervised and unsupervised CPD approaches were reviewed in context of the study by Xu et al. [3] and subsequently implemented based on insights obtained from the review.

1) Unsupervised Methods: PELT (Pruned Exact Linear Time) [4] is an offline, unsupervised CPD algorithm that finds an optimal set of change points by minimizing a penalized cost function over possible segmentations. In my experiments, PELT produced moderate segmentation quality. Out of 87 samples, 43 samples (**49.4%**) had the correct number of detected change points. The average overlap (Intersection over Union, IoU) between predicted and true segments was **0.7380**. The average detection delay for matched change points was **827.44 ms**. Figure 4 shows best and worst IoU overlaps for PELT. The best segmented sample had an IoU of 0.98 with a detection delay of 40ms. However, the worst segmented sample had an IoU of 0.49 with a delay of 1.9s, highlighting the need for exploring better CPD models.

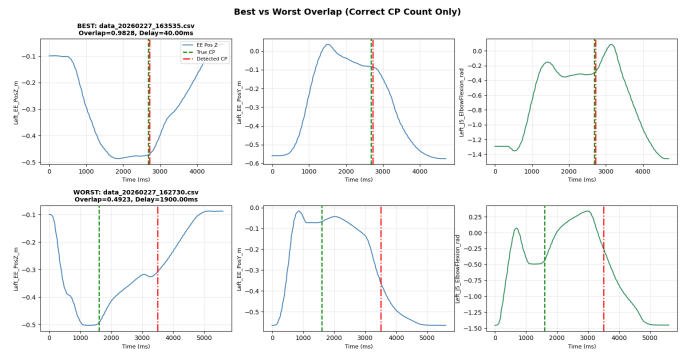

Fig. 4: Best and worst segmentation results with PELT

2) Supervised Methods: The supervised Change point Detector used in this project is a 1D convolutional neural network that predicts per time step probabilities for the presence of a change point. Inputs are fixed-length (interpolated to length of 500) and standardized time series. The network consists of three Conv1D layers, each followed by BatchNorm and ReLU with a 1×1 Conv1D output head.

Training minimizes a binary cross-entropy loss, optimized with Adam at learning rate 10^{-3}. The pipeline uses 5-fold cross validation, 50 epochs, batch size 4 and early stopping with patience 15. Data pre-processing applies StandardScaler and optional temporal augmentations (jitter, scaling, shift, warping, frame skipping).

Inference supports either selecting the top K peaks or thresholding predicted probabilities (default threshold = 0.5). Predicted and true change points are matched within a tolerance window (default $\tau = 20$ samples or $400ms$) to compute detection precision, recall and F1. This means that a True Positive value is recorded when the Predicted and True change points have a maximum difference given by the tolerance window. Segmentation quality is reported via per-state and average IoU.

During evaluation, the supervised CNN achieved an average IoU of **0.9400** (IoU for GOING = 0.9398, IoU for RETURNING = 0.9403) with a precision of **0.7778** ($\tau = 400ms$). Fig. 5 shows best and worst IoU overlaps for the CNN model. Figure 6 displays the precision versus tolerance window plot used to select a practical matching tolerance.

IV. Future Plans and Goals

Future work in this project will focus on experimenting with a wider range of machine learning models for change point detection. In particular, sequence-based models such as Long Short-Term Memory (LSTM) networks will be explored, as they are well suited for handling time-series data with variable sequence lengths. Investigating these models

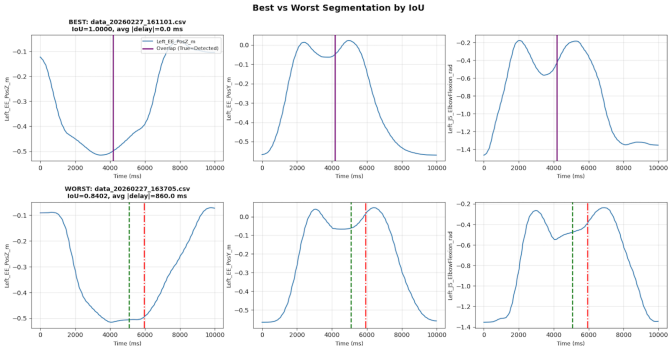

Fig. 5: Best and worst segmentation results with CNN based CPD

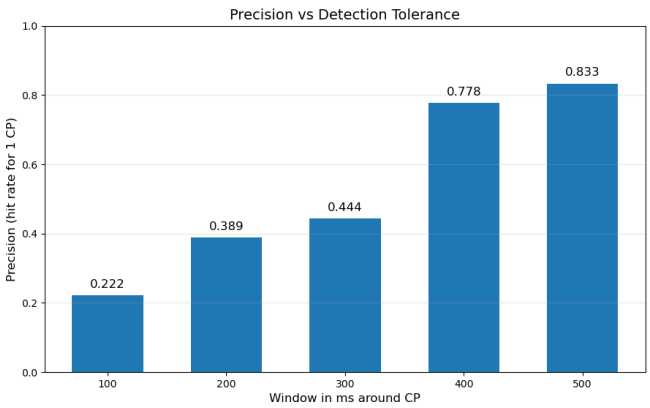

Fig. 6: Variation in Precision with tolerance window

may improve the robustness and accuracy of detecting state transitions within the recorded motion data.

At present, the dataset consists of a simplified task with two states and a single change point. In future stages of the project, additional data will be collected from more complex, multi-state tasks similar to those discussed in the background research section. This will allow the investigation of scenarios involving multiple change points, where both supervised and unsupervised detection methods can be applied and evaluated for more complex movement patterns.

V. Skill Development and Learning

Working on this project helped develop important soft skills alongside technical knowledge. The initial phase involved resolving multiple system-level issues related to Windows connectivity, Graphical User Interface (GUI) execution, and communication frequency. Troubleshooting these challenges strengthened my patience, perseverance, and systematic problem-solving ability. This process emphasized the importance of thoroughly evaluating all potential causes of an issue rather than adopting quick or incomplete solutions.

At the start, I had no experience with C# and Windows application development. Through this project, I learned to design and implement a fully functional GUI using Windows Forms in Microsoft Visual Studio. This included managing shared memory communication, CSV logging, and integrating robot hardware and software interfaces.

The project helped develop a foundational understanding of muscle physiology and human movement mechanics. I gained knowledge regarding muscle activation patterns, electromyography (EMG) signal acquisition, and functional electrical stimulation (FES) techniques. I understood the process of measuring voluntary muscle activation and the application of stimulation to assist impaired limb movement.

Although I had prior experience working with machine learning models, this project expanded my knowledge in temporal signal analysis. I developed confidence in applying machine learning techniques to rehabilitation movement data.

The project provided interdisciplinary exposure combining robotics, biomedical engineering, and data science. I developed skills in robotic system interaction, movement analysis, data acquisition, and exploratory application of computational models for rehabilitation research.

References

[1] C. L. LYNCH and M. R. POPOVIC, “Functional electrical stimulation,” *IEEE Control Systems Magazine*, vol. 28, no. 2, pp. 40–50, 2008. [Online]. Available: https://doi.org/10.1109/MCS.2007.914689

[2] F. Resquín, A. Cuesta-Gómez, J. Gonzalez-Vargas, F. Brunetti, D. Torricelli, F. Molina-Rueda, R. Cano de la Cuerda, J. Miangolarra, and J. Pons, “Hybrid robotic systems for upper limb rehabilitation after stroke: A review,” *Medical Engineering & Physics*, vol. 38, 09 2016. [Online]. Available: https://doi.org/10.1016/j.medengphy.2016.09.001

[3] R. Xu, Z. Song, J. WU, C. Wang, and S. Zhou, “Change-point detection with deep learning: A review,” *Frontiers of Engineering Management*, vol. 12, pp. 154–176, 02 2025. [Online]. Available: https://doi.org/10.1007/s42524-025-4109-z

[4] R. Killick, P. Fearnhead, and I. Eckley, “Optimal detection of changepoints with a linear computational cost,” *Journal of the American Statistical Association*, vol. 107, pp. 1590–1598, 12 2012. [Online]. Available: https://doi.org/10.1080/01621459.2012.737745